

15. Complessità computazionale



Lezione 31 - @December 3, 2025

Complessità computazionale

quanto è difficile il problema rispetto all'uso di risorse

Problema → Algoritmo di risoluzione

→ Risorse (tempo, spazio, ...) - stima delle risorse necessarie e sufficienti per risolvere il problema

Risorse necessarie e sufficienti

- r risorsa computazionale (tempo, spazio, ...)
- Π problema risolubile alitmicamente

Quanta risorsa r è necessaria e sufficiente per risolvere Π ?

Limitazione superiore - upper bound

$f : \mathbb{N} \rightarrow \mathbb{N}$ funzione

$f(n)$ risorsa r è **sufficiente** per risolvere Π se $\exists$ algoritmo a che risolve Π utilizzando su **ogni input** di lunghezza n **al più** $f(n)$ risorsa r (per ogni $n \geq 0$).

Limitazione inferiore - lower bound

$g : \mathbb{N} \rightarrow \mathbb{N}$ funzione

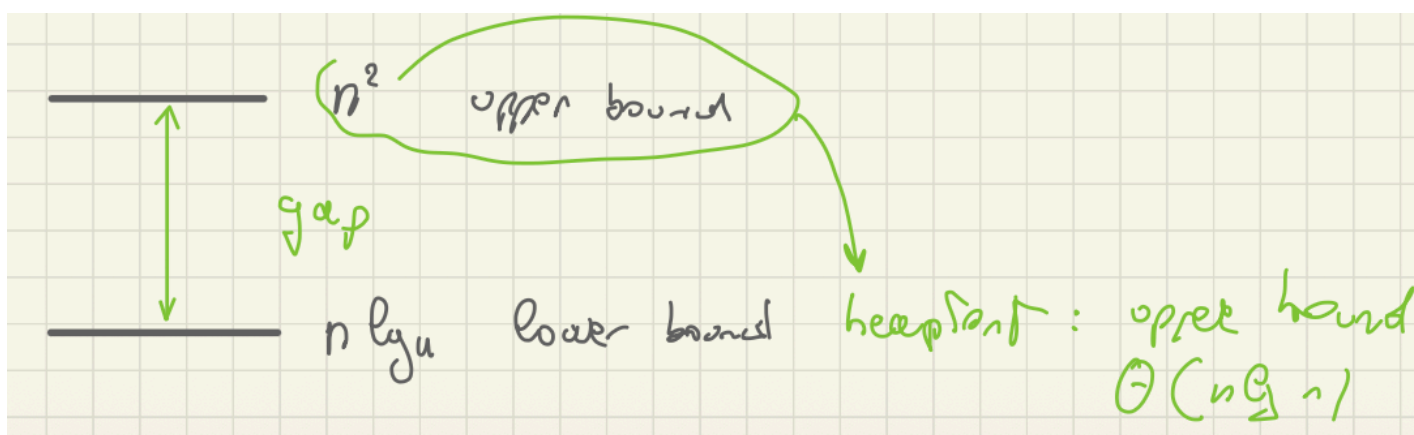
$g(n)$ risorsa r è **necessaria** per risolvere Π se $\forall$ algoritmo a che risolve Π **esiste** un input di lunghezza n (per ogni $n \geq 0$) sui cui a utilizza **almeno** $g(n)$ risorsa r

Esempio 1

Π = SORT

r = # confronti

- Algoritmo `insertionSort` $\Theta(n^2)$ confronti
- ogni algoritmo di ordinamento basato su confronti deve utilizzare un numero di confronti almeno dell'ordine di $n \log n$ nel caso peggiore → $\Omega(n \log n)$



posso avvicinare i bound (trovare algoritmi migliori o dimostrare che il lower bound è più alto)

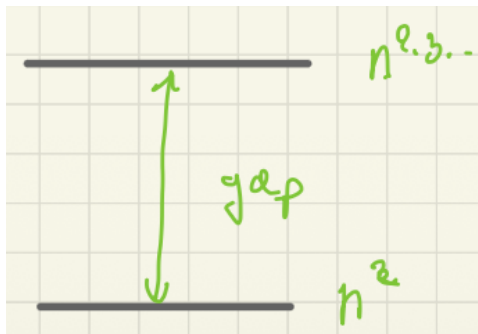
Esempio 2

Π = prodotto di matrici $n \times n$

r = #operazioni elementari (tempo)

- upper bound: $\Theta(n^3)$ prodotto righe $\times$ colonne

- $\Theta(n^{2.81})$ algoritmo di Strassen
- $\Theta(n^{2.3...})$ algoritmo migliore noto
- lower bound $\Omega(n^2)$ per vedere tutti gli elementi di ciascuna matrice



Classe di complessità

👉 Insieme di problemi che possono essere risolti utilizzando la stessa quantità di una determinata risorsa

Esempio - classe P

Classe P è la classe dei problemi risolubili in tempo polinomiale rispetto alla lunghezza dell'input.

- P ordinamento
- prodotto di matrici
- cammini minimi da un vertice a tutti gli altri
- ...

colorazione di grafi?

zaino?

commesso viaggiatore?

Problema

$$\Pi \subseteq I \times S$$

I universo delle possibile istanze (input)

S universo delle soluzioni

$$(x, s) \in \Pi \iff s \text{ è una soluzione di } \Pi \text{ su input } x$$

Esempio: Albero ricoprente

I = grafi non orientati

S = alberi

$$(x, s) \in \Pi \iff s \text{ è un albero ricoprente per il grafo } x$$

problema di ricerca

Esempio: Albero ricoprente minimo

I = grafi non orientati, S = alberi

$$(x, s) \in \Pi \iff s \text{ è un albero ricoprente per il grafo } x$$

minimo: tra tutti gli alberi s tc $(x, s) \in \Pi$ cerchiamo s^* tc $\text{peso}(p^*) \leq \text{peso}(p)$

problema di ottimizzazione

Tipologie di problemi

- **ricerca**

Data $x \in I$ trovare $s \in S : (x, s) \in \Pi$

• **ottimizzazione**

Data $x \in I$ trovare $s \in S : (x, s) \in \Pi$ che soddisfi un criterio di ottimalità fissato (es: min/max)

• **decisione** $S = \{0, 1\}$ risposta binaria

$(x, 1) \in \Pi$ istanze positive

$\Pi(x) = 1$ istanza positiva

$(x, 0) \in \Pi$ istanze negative

$\Pi(x) = 0$ istanza negativa

no contraddizioni

Esempi di problemi di decisione: albero ricoprente di peso limitato

- I = Grafi non orientati

istanza $x \in I$

questione x è connesso?

- I = Grafi non orientati pesati $\times \mathbb{N}$

istanza $(x, k) \in I$

questione esiste un albero ricoprente di x di peso $\leq k$?

Ottimizzazione vs decisione

esempio: albero ricoprente

ottimizzazione

Dato $G = \langle V, E, \omega \rangle$ grafo non orientato connesso pesato trovare un albero ricoprente di G di peso minimo

decisione

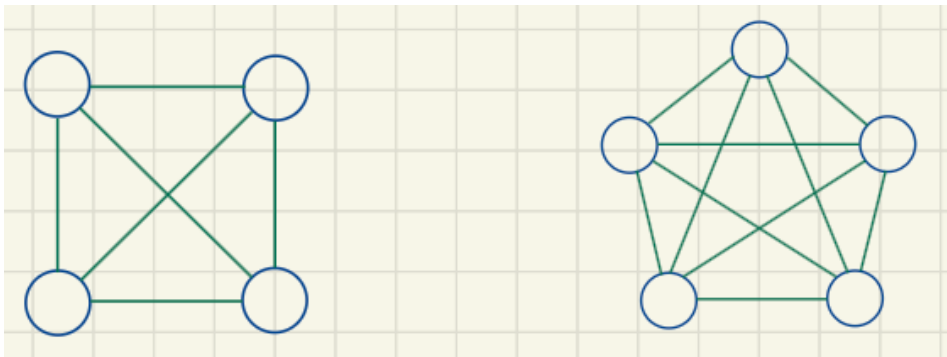
Dato $G = \langle V, E, \omega \rangle$ grafo non orientato connesso pesato e un intero $k \in \mathbb{N}$, esiste un albero ricoprente di G di peso $\leq k$?

👉 Se so risolvere un problema di ottimizzazione so anche risolvere un problema di decisione, molto spesso le complessità computazionali sono paragonabili.

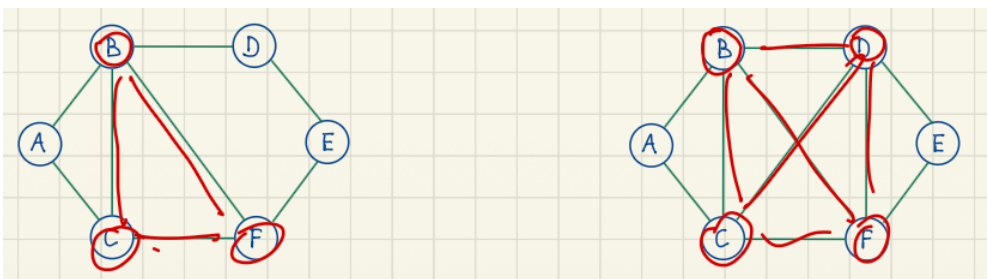
Problema della cricca (clique)

grafo non orientato completo

tutti i vertici sono connessi.



sottografo completo



Ottimizzazione vs decisione

ottimizzazione

Dato $G = \langle V, E, \omega \rangle$ grafo non orientato trovare un sottografo completo con il massimo numero di vertici

decisione

Dato $G = \langle V, E, \omega \rangle$ grafo non orientato e un intero $k \in \mathbb{N}$ stabilire se G contiene una clique di k vertici



al momento questo problema non ha soluzione.

Classi di complessità

$s, t : \mathbb{N} \rightarrow \mathbb{N}$ funzioni

- $TIME(t(n))$ = classe dei problemi di decisione risolubili da algoritmi che usano tempo $O(t(n))$.
- $SPACE(s(n))$ = classe dei problemi di decisione risolubili da algoritmi che usano spazio $O(s(n))$.

classe P

$$P = \bigcup_{c=0}^{\infty} TIME(n^c)$$

classe dei problemi di decisione risolubili in tempo polinomiale rispetto la lunghezza dell'input.

Altre classi

$$PSPACE = \bigcup_{c=0}^{\infty} TIME(n^c)$$

spazio polinomiale

$$EXPTIME = \bigcup_{c=0}^{\infty} TIME(2^{n^c})$$



Lezione 32 - @December 5, 2025

Problemi di decisione

1. non ci interessano i tempi di Input/Output, ma solo i tempi di calcolo della soluzione
2. input di dominio diverso, risposta sempre sì/no

Osservazioni sulle classi

$$P \subseteq EXPTIME$$

Relazioni spazio/tempo

- **in tempo t quanto spazio riusciamo a utilizzare?**
 - Ogni istruzione macchina utilizza al massimo k celle, con k fissato
 - il numero di celle di memoria utilizzabili in tempo t è $\leq kt$
- se tempo è polinomiale → spazio polinomiale

$$t(n), \quad spazio \leq kt(n)$$

$$\Rightarrow P \subseteq PSPACE$$

- **in spazio s quanto tempo posso utilizzare senza entrare in loop?**
considerando algoritmi che terminano.

```
x,y,z boolean
while (x or y != z) {
    ....
}
```

il numero complessivo di possibilità sono $2^3 \rightarrow$ se supero questo limite sono in un loop.

Se la memoria ha spazio fissato: s celle, dove ogni cella ha posizione fissata di k bit ciascuna $\rightarrow 2^k$ configurazioni per ogni cella

$$(2^k)^s = 2^{k \cdot s} \text{ possibili contenuti della memoria}$$

In un programma $\mathcal{P}$ formato da p istruzioni, se eseguo la stessa istruzione con lo stesso contenuto della memoria $\rightarrow$ loop infinito:

- $2^{k \cdot s}$ contenuti della memoria
- p possibili istruzioni

$$t > 2^{k \cdot s} \implies \text{loop infinito}$$

programmi che terminano $t \leq 2^{k \cdot s} \cdot p$

spazio polinomiale $\implies$ tempo al più esponenziale

$$PSPACE \subseteq EXPTIME$$

Quindi:

$$P \subseteq PSPACE \subseteq EXPTIME$$

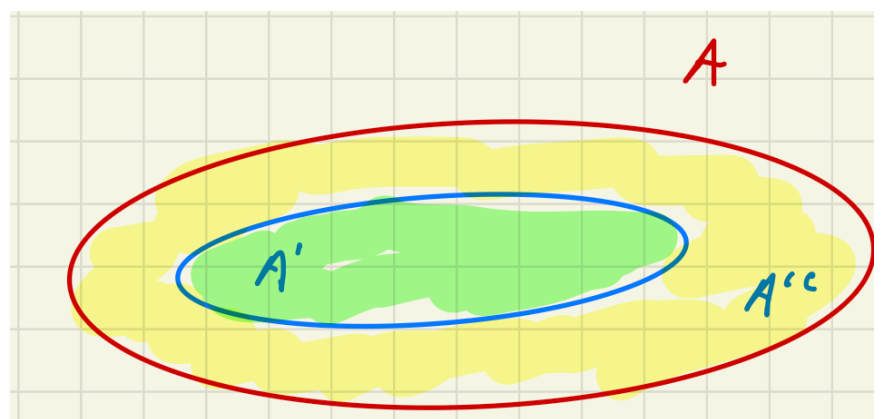
- $P \subset EXPTIME$ inclusione propria
- almeno una delle due $\subseteq$ è un inclusione propria ma non si sa quale.

Esempi di problemi

Partizione

istanza

Insieme finito A di oggetti, funzione peso $s : A \rightarrow \mathbb{N}$



questione

$$\exists A' \subseteq A : \sum_{a \in A'} s(a) = \sum_{a \in A \setminus A'} s(a)?$$

provare tutti i sottoinsiemi. per n elementi 2^n sottoinsiemi

Clique

istanza

Grafo non orientato $G = (V, E)$, intero $k \geq 0$

questione

$\exists$ sottografo completo di G con k vertici?

per provare tutti i vertici:

$$\binom{n}{k} \approx n^k$$

Formule booleane in forma normale congiuntiva

- V insieme finito di variabili $x, y, z, \dots$
- **LETTERALE** variabile o variabile negata $x, \overline{x}$
- **CLAUSOLA** disgiunzione (**or**) di letterali $x \vee \overline{y} \vee \overline{z}$
- **FORMULA** congiunzione di clausole

$$\phi(x, y, z) = (x \vee \overline{y} \vee \overline{z}) \wedge (\overline{x} \vee \overline{y}) \wedge (\overline{x} \vee \overline{z})$$

- **ASSEGNAMENTO** $f : V \rightarrow \{0, 1\}$
 $\phi(f)$ valore della formula f assegnando a ogni variabile x il valore $f(x)$

Problema della soddisfacibilit  SODD

istanza

formula booleana ϕ in forma normale congiuntiva con insieme di variabili V

questione

$\exists$ un assegnamento alle variabili in V che rende vera ϕ , cio  tc $\phi(f) = 1$?

Esempio

$$\phi = (x \vee z) \wedge (\overline{x} \vee y \vee z) \wedge (x \vee y \vee \overline{z})$$

	f	
x	1	$\rightarrow \phi$ � soddisfacibile
y	1	
z	0	

$$\phi(f) = 1$$

$$\phi = (x \vee \overline{y}) \wedge \overline{x} \wedge \overline{y}$$

x	y	
0	0	0
0	1	0
1	0	0
1	1	0

SODD

- $\phi(x_1, x_2, \dots, x_n)$ formula con n variabili
- per decidere se ϕ   soddisfacibile, proviamo tutti i possibili assegnamenti di valori alle variabili $x_1, x_2, \dots, x_n$

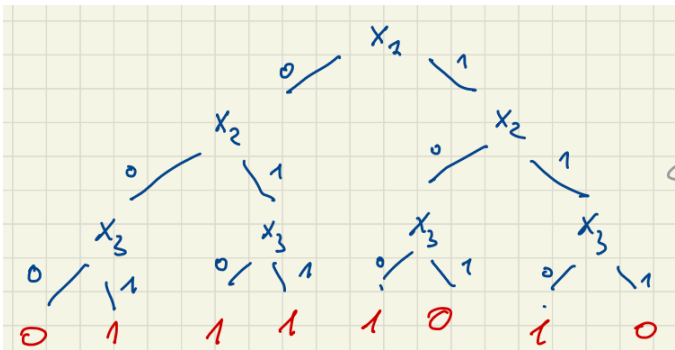
$$\begin{aligned} \# \text{assegnamenti possibili } & 2^n \\ \text{Tempo } & O(2^n) \\ \text{SODD} & \in EXPTIME \end{aligned}$$

Algoritmi non deterministici

- istruzione - indovina $b \in \{0, 1\}$
- tra le possibili scelte l’algoritmo indovina sempre quella “giusta” (se c’ )

```
ALGORITMO sodd (Formula  $\phi(x_1, \dots, x_n)$ )  $\rightarrow$  boolean
FOR  $i \leftarrow 1$  TO  $n$  DO
     $z_i \leftarrow$  indovina un valore in  $\{0, 1\}$ 
 $r \leftarrow \phi(z_1, z_2, \dots, z_n)$ 
RETURN  $r$ 
```

albero di computazione



Classe NP

classe dei problemi di decisione risolubili in tempo polinomiale da algoritmi non deterministici
→ classe dei problemi di decisione con certificati verificabili in tempo polinomiale



NP Non deterministico Polinomiale

Problema PARTIZIONE

istanza

Insieme finito A di oggetti, fz peso $s : A \rightarrow \mathbb{N}$

questione

$$\exists A' \subseteq A : \sum_{a \in A'} s(a) = \sum_{a \in A \setminus A'} s(a)?$$

```
ALGORITMO partizione (Array  $S[0, \dots, 1]$ )  $\rightarrow$  funzione
Sia  $b[1 \dots n]$  un array di  $\{0, 1\}$ 
FOR  $i \leftarrow 1$  TO  $n$  DO
     $b[i] \leftarrow$  indovina un valore in  $\{0, 1\}$ 
sommaScelti  $\leftarrow 0$ 
sommaAltri  $\leftarrow 0$ 
FOR  $i \leftarrow 1$  TO  $n$  DO
    IF  $b[i] = 1$  THEN sommaScelti  $\leftarrow$  sommaScelti +  $S[i]$ 
    ELSE sommaAltri  $\leftarrow$  sommaAltri +  $S[i]$ 
IF sommaScelti = sommaAltri THEN
    RETURN true
ELSE RETURN false
```

$$PARTIZIONE \in NP$$

Problema CLIQUE

Clique

istanza

Grafo non orientato $G = (V, E)$, intero $k \geq 0$

questione

$\exists$ sottografo completo di G con k vertici?

```
ALGORITMO clique (Grafo  $G = (V, E)$ , intero  $k$ )  $\rightarrow$  boolean
   $V' \leftarrow \emptyset$ 
  FOR EACH  $v \in V$  DO
     $b \leftarrow$  indovina un valore in  $\{0, 1\}$ 
  IF  $\#V' \neq k$  THEN
    RETURN false
  risposta  $\leftarrow$  true
  FOR EACH  $v \in V'$  DO
    FOR EACH  $u \in V'$  DO
      IF  $u \neq v$  AND  $(u, v) \notin E$  THEN
        risposta  $\leftarrow$  false
  RETURN risposta
```

$$\Rightarrow \text{CLIQUE} \in NP$$

Problemi NP

Algoritmo:

1. fase non deterministica $\rightarrow$ costruzione del certificato
2. fase deterministica $\rightarrow$ verifica del certificato in tempo polinomiale

$$P \subseteq NP$$

? $P \neq NP$

problemi NP – *completi*, se si riuscisse a trovare un algoritmo polinomiale per uno di questi problemi $\rightarrow P = NP$



Lezione 33 - @December 10, 2025



quando si nega un problema in NP si deve testare $\forall$, esempio SODD.

$$P \stackrel{?}{=} NP$$

NPSPACE

Classe dei problemi di decisione risolubili in spazio polinomiale da algoritmi nondeterministici.

$$PSPACE = NPSPACE$$

È stato dimostrato dal Teorema di Savitch (1970) - riciclo dello spazio - divide-et-impera

$\rightarrow$ "

$\rightarrow$ "

$$\Rightarrow P \subseteq NP \subseteq PSPACE \subseteq EXPTIME$$

$P \neq NP$?

https://excalidraw.com/#json=6xOo2rzTKGPD7VCWSL_5q,cNDAVijb5LPpzgezj8D0qw

Se $P \neq NP$?

- Clique $\in NP, \notin P$
- SODD $\in NP, \notin P$
- PARTIZIONE $\in NP, \notin P$
- GRAFO CONNESSO $\in P$
- ALBERO RICOPRENTE MINIMO $\in P$

Problemi NP -completi

problemi “più difficili” in NP

Se riesco a dimostrare che uno di loro non sta in P , tutti gli altri $\notin P \Rightarrow P \neq NP$

Se invece riesco a dimostrare che uno di loro $\in P$, tutti sono $\in P \Rightarrow P = NP$

NP-completezza

Insieme indipendente

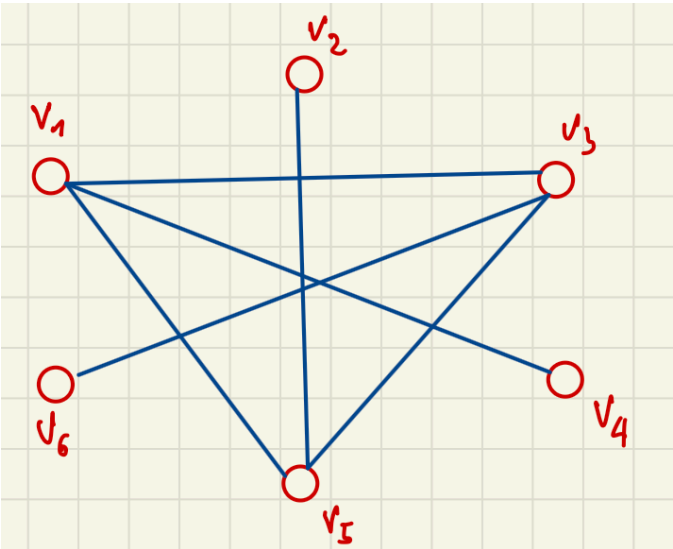
istanza

Grafo non orientato $G = (V, E)$

Intero $k \geq 0$

questione

$\exists V' \subseteq V$ con $\#V' = k$ tc i vertici in V' non hanno archi che li collegano direttamente (cioè se $u, v \in V'$ e $u \neq v$ allora $(u, v) \notin E$)

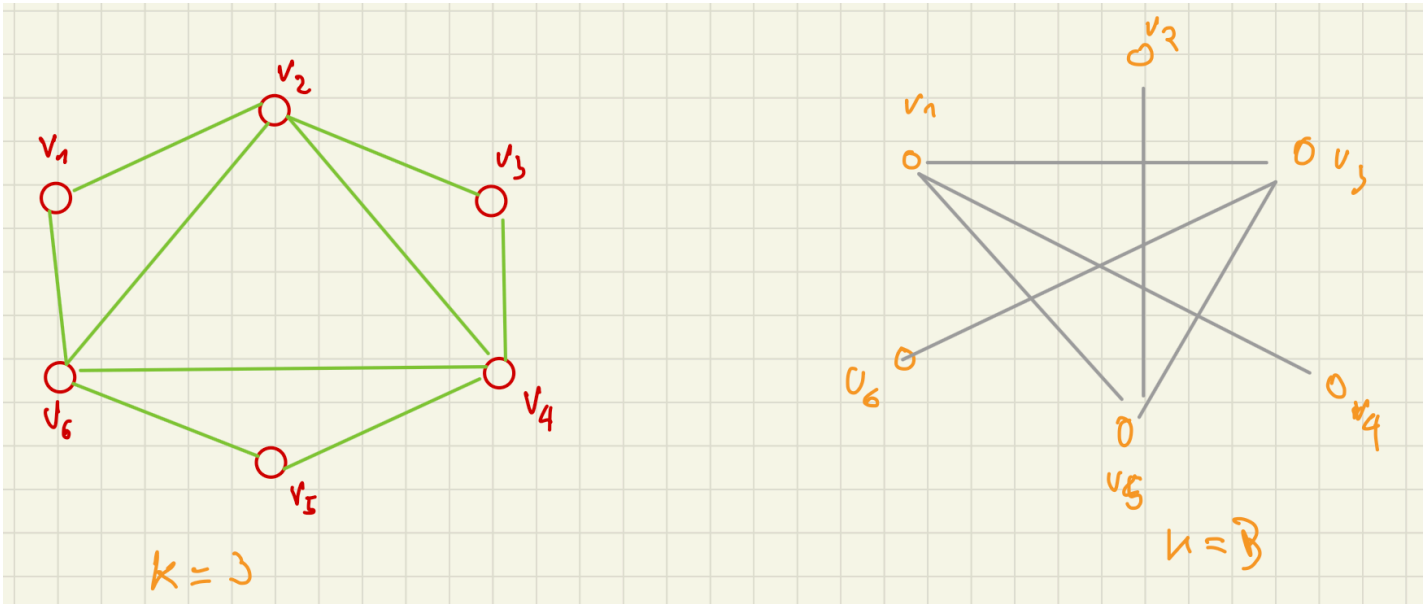


Da Clique a Insieme indipendente



Dato lo stesso insieme di vertici, si “negano” le posizioni degli archi
Una clique è a sua volta un insieme indipendente e viceversa. (Per istanze di grafo diverse con stessi vertici)

Problema	CLIQUE	INSIEME INDIPENDENTE
Istanza	Grafo non orientato $G = (V, E)$; intero $k \geq 0$	Grafo non orientato $G = (V, E)$; intero $k \geq 0$
Questione	$\exists$ sottografo completo di G con k vertici?	$\exists V' \subseteq V$ con $\#V' = k$ tc i vertici in V' non hanno archi che li collegano direttamente?

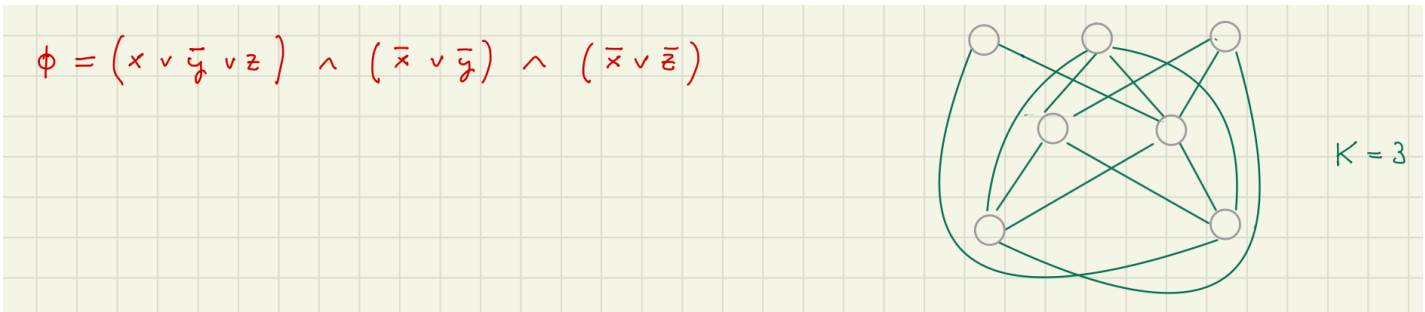


Da SODD a Clique



ogni clique corrisponde ad un assegnamento che valida la formula.

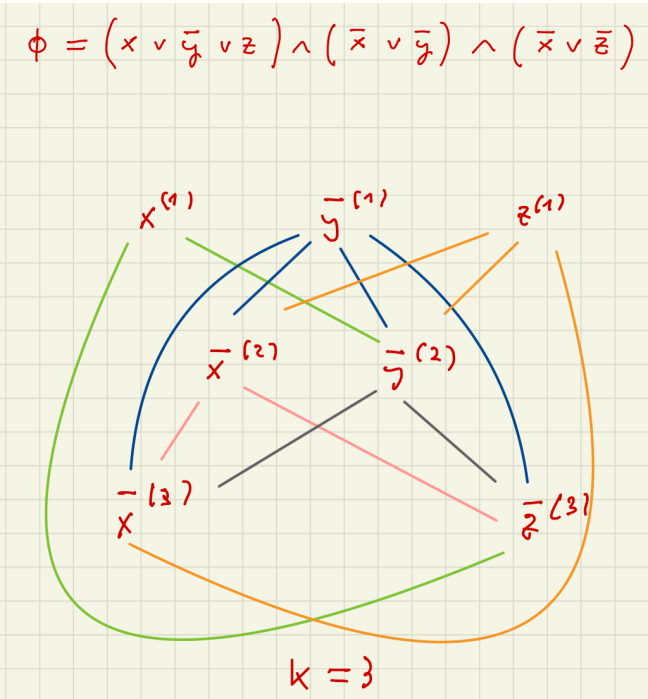
Problema	SODD	CLIQUE
Istanza	Formula booleana ϕ in forma normale congiuntiva con insieme di variabili V	Grafo non orientato $G = (V, E)$; intero $k \geq 0$
Questione	$\exists$ un assegnamento alle variabili in V che rende vera ϕ , cioè $\phi(f) = 1$?	$\exists$ sottografo completo di G con k vertici?



ϕ formula in forma normale congiunta

$\Downarrow$

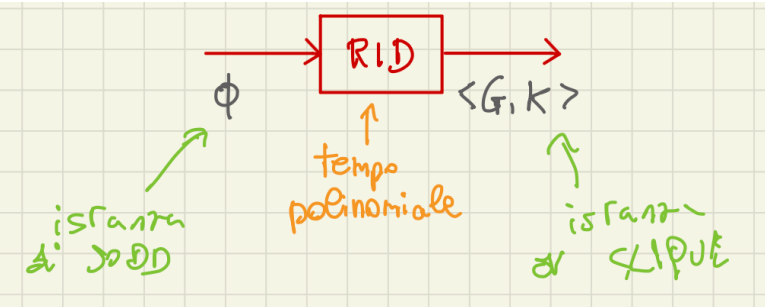
- $G = (V, E)$
- $V = \{\text{occorrenze di letterali in } \phi\}$
- $(u, v) \in E \iff u \text{ e } v \text{ sono in clausole differenti e non sono uno il negato dell'altro}$
- $k = \# \text{clausole di } G$



in altre parole

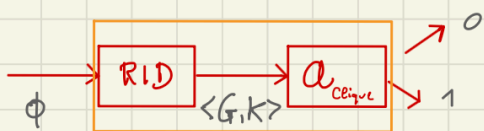
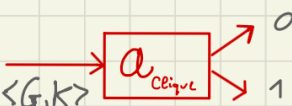
$\exists$ algoritmo RID che data in ingresso una formula ϕ in forma normale congiunta, in **tempo polinomiale** restituisce una coppia $\langle G, k \rangle$ (grafo, intero) tc:

ϕ è soddisfacibile $\iff G$ contiene una clique di k vertici



da SODD a CLIQUE

ALGORITMO che risolve CLIQUE



a_{SODD}

risolve SODD !

ALGORITMO $a_{SODD}(\phi \text{ formula}) \rightarrow \text{boolean}$
 $\langle G, k \rangle \leftarrow RID(\phi)$
 $r \leftarrow a_{CLIQUE}(\langle G, k \rangle)$
 RETURN r

Inoltre:

Se a_{CLIQUE} lavora in tempo polinomiale
 anche a_{SODD} lavora in tempo polinomiale

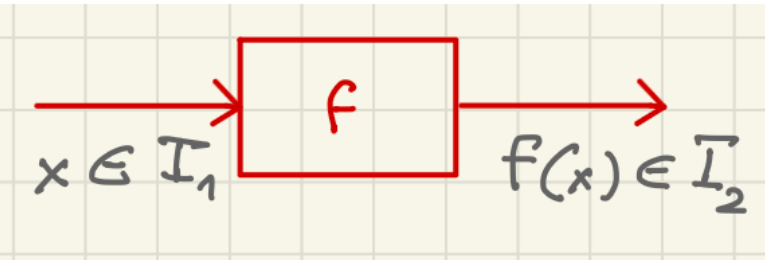
Se $CLIQUE \in P$
 allora $SODD \in P$

Riducibilità tra problemi

$\Pi_1 : I_1 \rightarrow \{0, 1\}$ problemi di decisone

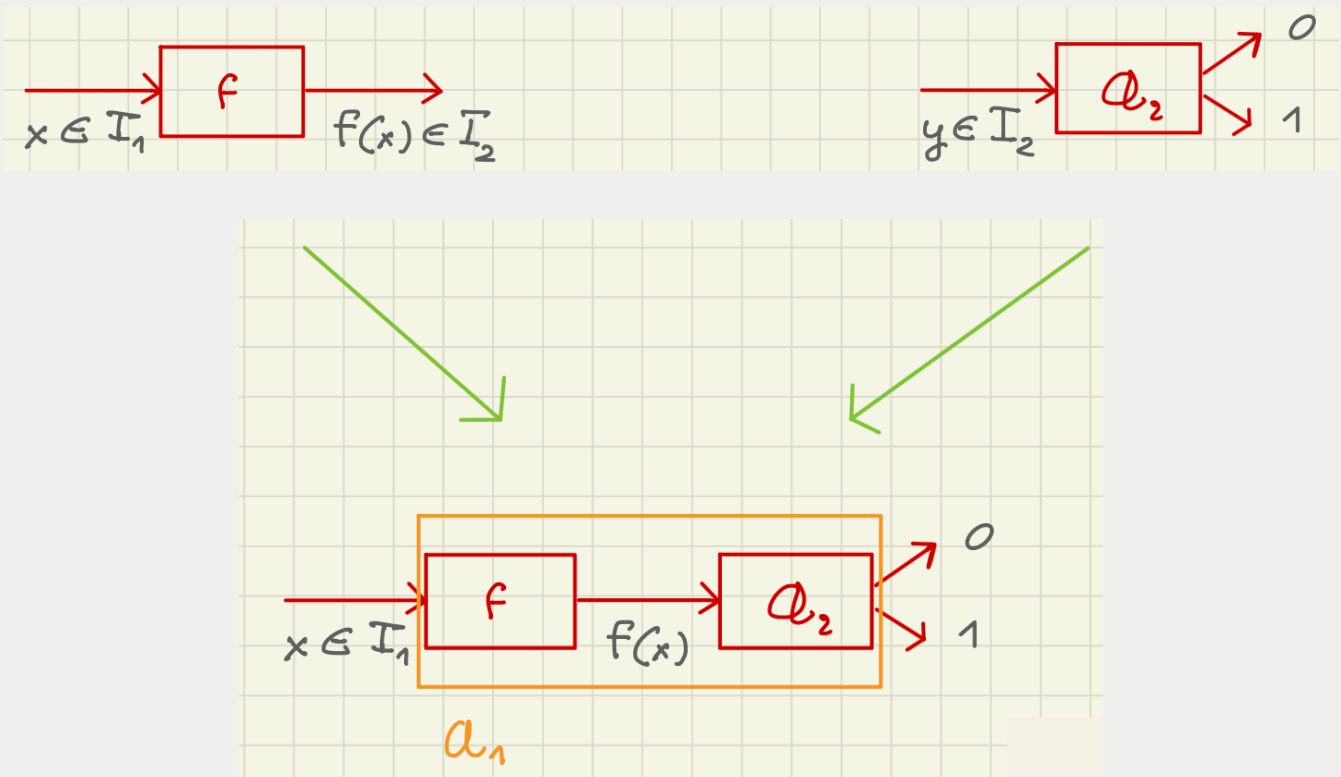
$\Pi_2 : I_2 \rightarrow \{0, 1\}$

Π_1 è riducibile a Π_2 se $\exists F : I_1 \rightarrow I_2 : \forall x \in I_1 \quad \Pi_1(x) = \Pi_2(F(x))$



Supponiamo

- a. Π_1 riducibile a Π_2 b. $\exists$ algoritmo a_2 per Π_2

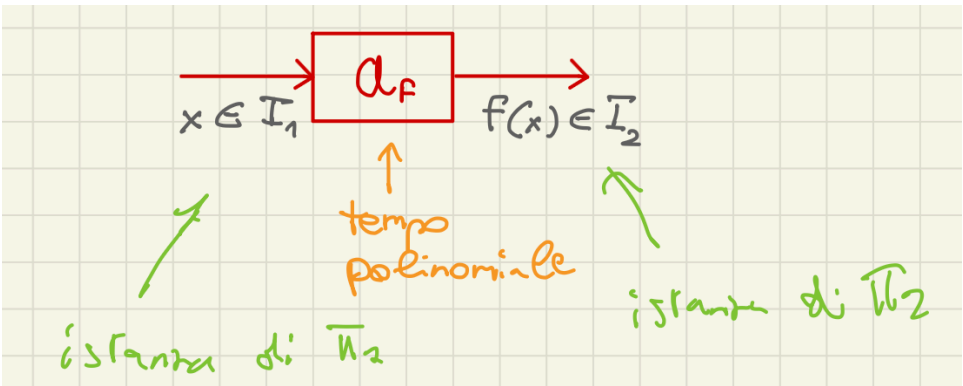


ALGORITMO $a_1(x \in I_1) \rightarrow \text{boolean}$
 $y \leftarrow f(x)$
 $r \leftarrow a_2(y)$
 RETURN r

a_1 è un algoritmo per Π_1

Riducibilità polinomiale

Π_1 è riducibile **polinomialmente** a Π_2 ($\Pi_1 \leq_P \Pi_2$) se $\exists$ riduzione $f : I_1 \rightarrow I_2$ calcolare da un algoritmo a_f in tempo polinomiale

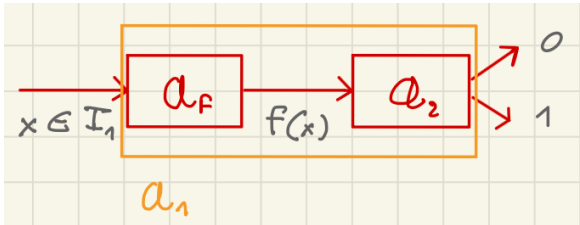


ALGORITMO $a_1(x \in I_1) \rightarrow \text{boolean}$
 $y \leftarrow a_f(x)$ **tempo** $O(p(|x|))$
 $r \leftarrow a_2(y)$ **tempo** $O(q(|y|))$
 RETURN r

Se a_f e a_2 lavorano in tempo polinomiale, in quanto tempo lavora a_1 ?
In tempo $p(n)$ produco output di lunghezza $O(p(n)) \rightarrow |y| = O(p(|x|))$
tempo totale: $O(p(|x|) + q(|y|)) = O(p(|x|) + q(p(|x|))) = O(poly(|x|))$

Proprietà fondamentali per la riducibilità polinomiale

1. Se $\Pi_1 \leq_P \Pi_2$ e $\Pi_2 \in P$ allora $\Pi_1 \in P$



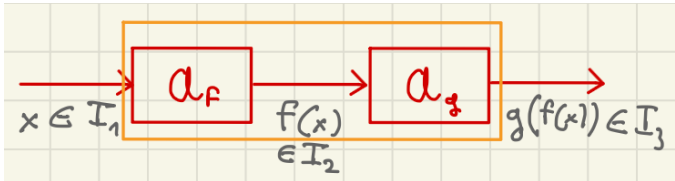
esempio:

$$\Pi_1 = SODD \quad \Pi_2 = CLIQUE$$

$$SODD \leq_P CLIQUE$$

$$\text{Se } CLIQUE \in P \Rightarrow SODD \in P$$

$$2. \text{ Se } \Pi_1 \leq_P \Pi_2 \text{ e } \Pi_2 \leq_P \Pi_3 \Rightarrow \Pi_1 \leq_P \Pi_3$$



esempio:

$$SODD \leq_P CLIQUE$$

$$CLIQUE \leq_P INSIEME\ INDIPENDENTE$$

$$\rightarrow SODD \leq_P INSIEME\ INDIPENDENTE$$

Problemi NP-hard e NP-completi

- Π è NP-hard (NP-difficile) se $\forall \Pi' \in NP : \Pi' \leq_P \Pi$ (Π' è riducibile polinomialmente a Π)
- Π è NP-completo se:
 - Π è NP-hard
 - $\Pi \in NP$

→ Π è un problema “più difficile” tra tutti i problemi in NP.

Teorema

Sia Π un problema NP-completo

- se $\Pi \notin P \Rightarrow P \neq NP$
- se $\Pi \in P \Rightarrow P = NP$

Teorema di Cook

SODD è NP-completo

dim (idea)

1. $SODD \in NP$
2. $\forall \Pi' \in NP \quad \forall \Pi' \leq_P SODD$

CLIQUE è NP-completo

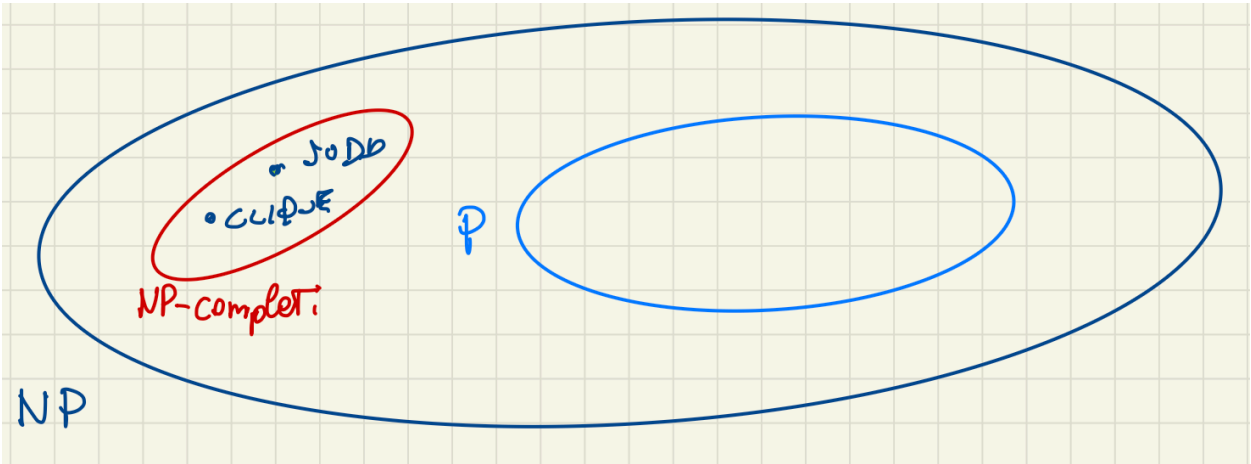
dim (idea)

1. $CLIQUE \in NP$
2. Abbiamo visto che $SODD \leq_P CLIQUE$, poiché SODD è NP-completo: $\forall \Pi' \in P \quad \Pi' \leq_P SODD$
 → $\Pi' \leq_P CLIQUE$ → CLIQUE è NP-hard → CLIQUE è NP-completo

in generale

Se Π è NP-completo e $\Pi \leq_P \tilde{\Pi}$ con $\tilde{\Pi} \in P$ allora $\tilde{\Pi}$ è NP-completo

Se $P \neq NP$



Lezione 34 - @December 12, 2025

Alcuni problemi NP-completi

1. Cammino Hamiltoniano

istanza

Grafo $G = (V, E)$

questione

$\exists$ cammino che visita tutti i nodi del grafo una e una sola volta?

2. Cammino più lungo

istanza

Grafo $G = (V, E, \omega)$ grafo pesato

$k \in \mathbb{N}$

questione

$\exists$ cammino semplice in G di lunghezza $\geq k$?

cammino hamiltoniano

$$G = (V, E) \Rightarrow$$

cammino più lungo

$$\begin{aligned} G &= (V, E, \omega) \\ \omega(x, y) &= 1 \quad \forall (x, y) \in E \\ k &= n - 1 \end{aligned}$$

3. SODD

istanza

Formula booleana ϕ in forma normale con clausole di lunghezza 3?

questione

ϕ è soddisfacibile?

$$x \vee y \vee z \vee w \equiv (x \vee y \vee a) \wedge (\bar{a} \vee z \vee w)$$

$$x \vee y \vee z \vee w \vee k \equiv (x \vee y \vee a) \wedge (\bar{a} \vee z \vee b) \wedge (\bar{b} \vee w \vee k)$$

3-SODD è **NP-completo**:

2-SODD: $(x \vee \bar{y}) \wedge (x \vee \bar{z}) \wedge \dots \in P$

f.n. disgiunta $(x \wedge \bar{y} \wedge z) \vee (\bar{x} \wedge \bar{y}) \vee (x \wedge \bar{z} \wedge y) \in P$



SODD $\rightarrow$ DSODD costo?

4. Partizione

istanza

Insieme finito A di oggetti

Funzione peso $s : A \rightarrow \mathbb{N}$

questione

»?

5. Commesso viaggiatore

istanza

$C = \{c_1, c_2, \dots, c_n\}$ insieme finito di città $d(c_i, c_j) \in \mathbb{Z}^+$ distanze tra coppie di città $i, j = 1, \dots, m$

$B \in \mathbb{Z}^+$ numero

questione

$\exists$ "tour" che visita tutte le città di lunghezza al più B ?

